



## รายงานปฏิบัติการ

### PS04: Memory class and API in C programming

#### เสนอ

อาจารย์ ราชวิรัช สโรชวิกสิต

#### จัดทำโดย

|                           |             |
|---------------------------|-------------|
| นายภูมิพัฒน์ อภิวัตนะพงศ์ | 67070501035 |
| นายวิรัช ทองอุทัยศรี      | 67070501041 |
| นายเจษฎา เกียรติกมลวงค์   | 67070501080 |

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

# Contents

|                                                                         |          |
|-------------------------------------------------------------------------|----------|
| สภาพแวดล้อมที่ใช้ในการทดลอง                                             | 1        |
| <b>1    ข้อ 1: Static Storage Class</b>                                 | <b>1</b> |
| 1.1    หลักการพื้นฐานของ Storage Class                                  | 1        |
| 1.2    Step 1: โค้ดโปรแกรมที่มี static (คอมไพล์แบบปกติ)                 | 1        |
| 1.2.1    ผลการทดลอง                                                     | 2        |
| 1.2.2    การวิเคราะห์ผลการทดลอง                                         | 2        |
| 1.3    Step 2-3: โค้ดโปรแกรมที่ลบคำว่า static ออก                       | 2        |
| 1.3.1    ผลการทดลอง                                                     | 3        |
| 1.3.2    การวิเคราะห์ผลและเปรียบเทียบกับ Step 1                         | 3        |
| 1.4    Step 4: การคอมไพล์ด้วยแฟล็ก -no-pie และการเปรียบเทียบ            | 3        |
| 1.4.1    ผลการทดลอง                                                     | 3        |
| 1.4.2    วิเคราะห์ความแตกต่างจากเดิมและสาเหตุ                           | 4        |
| 1.5    สรุปผลการทดลองข้อ 1                                              | 4        |
| <b>2    ข้อ 2: Extern Storage Class</b>                                 | <b>4</b> |
| 2.1    หลักการพื้นฐาน: การนิยาม (Definition) และการประกาศ (Declaration) | 4        |
| 2.2    Step 1: โค้ดโปรแกรมที่มี extern ทั้งสองฟังก์ชัน                  | 5        |
| 2.2.1    ผลการทดลอง                                                     | 5        |
| 2.2.2    การวิเคราะห์ผลการทดลอง                                         | 5        |
| 2.3    Step 2-3: โค้ดโปรแกรมที่ลบคำว่า extern ในฟังก์ชัน main()         | 6        |
| 2.3.1    ผลการทดลอง                                                     | 6        |
| 2.3.2    การวิเคราะห์ผลและเปรียบเทียบกับ Step 1                         | 6        |
| 2.4    Step 4: การคอมไพล์ด้วยแฟล็ก -no-pie และการเปรียบเทียบ            | 7        |
| 2.4.1    วิเคราะห์ความแตกต่างจากเดิมและสาเหตุ                           | 7        |
| 2.5    สรุปผลการทดลองข้อ 2                                              | 7        |
| <b>3    ข้อ 3: การจัดสรรหน่วยความจำด้วย malloc, realloc และ free</b>    | <b>7</b> |
| 3.1    หลักการพื้นฐานของการจัดสรรหน่วยความจำ (Stack vs Heap)            | 8        |
| 3.2    Step 1-3: ผลการทดลองและการวิเคราะห์โค้ดเริ่มต้น                  | 8        |
| 3.2.1    ผลการทดลอง                                                     | 9        |
| 3.2.2    การวิเคราะห์ตามแต่ละส่วนของผลลัพธ์ (Step 3)                    | 9        |
| 3.3    Step 4-6: ผลการทดลองเมื่อเปิดคอมเมนต์จอตัวแปร b                  | 10       |
| 3.3.1    ผลการทดลอง                                                     | 10       |
| 3.3.2    การวิเคราะห์และเปรียบเทียบกับรอบก่อนหน้า (Step 6)              | 10       |
| 3.4    สรุปผลการทดลองข้อ 3                                              | 11       |

## สภาพแวดล้อมที่ใช้ในการทดลอง

การทดลองทั้งหมดดำเนินการบนระบบปฏิบัติการตามข้อกำหนดของปฏิบัติการ โดยมีรายละเอียดสภาพแวดล้อมดังนี้

Table 1: ข้อมูลสภาพแวดล้อมของระบบที่ใช้ในการทดลอง

| รายการ         | รายละเอียด                                                                         |
|----------------|------------------------------------------------------------------------------------|
| ระบบปฏิบัติการ | Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11), Kernel 6.6.87.2-microsoft-standard-WSL2 |
| สถาปัตยกรรม    | x86-64 (64-bit)                                                                    |
| คอมไพเลอร์     | gcc 13.3.0 (ตั้งค่าเริ่มต้นเป็น --enable-default-pie)                              |
| ค่า ASLR       | /proc/sys/kernel/randomize_va_space = 2 (เปิดการสุ่มอย่างสมบูรณ์)                  |
| Shell          | GNU bash 5.2.21(1)-release                                                         |

**หมายเหตุ:** การคอมไพล์โปรแกรมด้วยแฟล็ก `-Wall -Wextra` ตามมาตรฐานจะพบ Warning `return type of 'main' is not 'int' [-Wmain]` เนื่องจากโค้ดต้นฉบับจากใบงานใช้ `void main()` ซึ่งในรายงานนี้ได้คงโค้ดไว้ตามข้อกำหนดเดิมของใบงานทุกประการ

## 1 ข้อ 1: Static Storage Class

### 1.1 หลักการพื้นฐานของ Storage Class

Storage Class ในภาษา C กำหนดคุณสมบัติสำคัญ 3 ประการของตัวแปร ได้แก่

- **ขอบเขต (Scope):** บริเวณในโปรแกรมที่สามารถเข้าถึงชื่อตัวแปรนั้นได้
- **อายุการใช้งาน (Lifetime):** ช่วงเวลาที่ตัวแปรคงอยู่ในหน่วยความจำระหว่างการทำงาน
- **ตำแหน่งในหน่วยความจำ (Memory Segment):** บริเวณที่ระบบจัดสรรพื้นที่ให้ เช่น Stack หรือ Data Segment

Table 2: เปรียบเทียบ Storage Class รูปแบบต่าง ๆ ในภาษา C

| การประกาศ                                | ขอบเขต      | อายุการใช้งาน             | ตำแหน่งจัดเก็บ            | ค่าเริ่มต้นโดยปริยาย   |
|------------------------------------------|-------------|---------------------------|---------------------------|------------------------|
| <code>int y;</code> ในบล็อก (auto)       | ภายในบล็อก  | ชั่วคราว (เข้า/ออก บล็อก) | Stack                     | ค่าขยะ (Indeterminate) |
| <code>static int y;</code> ในบล็อก       | ภายในบล็อก  | ตลอดการทำงานของโปรแกรม    | <code>.data / .bss</code> | 0                      |
| <code>static int y;</code> นอกฟังก์ชัน   | ภายในไฟล์   | ตลอดการทำงานของโปรแกรม    | <code>.data / .bss</code> | 0                      |
| <code>int y;</code> นอกฟังก์ชัน (Global) | ทั้งโปรแกรม | ตลอดการทำงานของโปรแกรม    | <code>.data / .bss</code> | 0                      |

ทั้งนี้ คำว่า `auto` ใน `auto int x = 3;` เป็นเพียงการระบุคลาสอัตโนมัติอย่างชัดเจน ซึ่งเป็นค่าเริ่มต้นของตัวแปรภายในฟังก์ชันอยู่แล้ว

### 1.2 Step 1: โค้ดโปรแกรมที่มี static (คอมไพล์แบบปกติ)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void main()
```

```

5 {
6     auto int x = 3;
7
8     while (x > 0)
9     {
10         static int y = 5;
11         y++;
12
13         printf("The value of y is %d\n", y);
14         printf("The address of y is %p\n\n", &y);
15
16         x--;
17     }
18 }

```

Code 1: src/q1\_static.c โค้ดเริ่มต้นตามใบงาน

### 1.2.1 ผลการทดลอง

ผลการคอมไพล์และรันโปรแกรม 3 ครั้ง:

```

$ gcc -Wall -Wextra -o q1_static q1_static.c
q1_static.c: In function 'main':
q1_static.c:10:6: warning: return type of 'main' is not 'int' [-Wmain]

$ ./q1_static                                     # Run 1
The value of y is 6      The address of y is 0x5c5469622010
The value of y is 7      The address of y is 0x5c5469622010
The value of y is 8      The address of y is 0x5c5469622010

$ ./q1_static                                     # Run 2
The value of y is 6      The address of y is 0x60a5f69ee010
The value of y is 7      The address of y is 0x60a5f69ee010
The value of y is 8      The address of y is 0x60a5f69ee010

$ ./q1_static                                     # Run 3
The value of y is 6      The address of y is 0x5faa56b46010
The value of y is 7      The address of y is 0x5faa56b46010
The value of y is 8      The address of y is 0x5faa56b46010

```

Code 2: ผลการทดลอง Step 1 (รัน 3 ครั้ง)

### 1.2.2 การวิเคราะห์ผลการทดลอง

- **ค่าของตัวแปร ( $y = 6 \rightarrow 7 \rightarrow 8$ ):** ตัวแปร static ถูกจัดสรรไว้ใน Data Segment (.data) และได้รับการกำหนดค่าเริ่มต้น (5) เพียงครั้งเดียวตั้งแต่ขั้นตอนคอมไพล์และโหลดโปรแกรม เมื่อวนลูปคำสั่งประกาศจะไม่ถูกทำงานซ้ำ คำสั่ง `y++` จึงบวกค่าสะสมต่อเนื่องจากรอบก่อนหน้า
- **ที่อยู่ของตัวแปรภายในการรันเดียวกัน:** มีตำแหน่งคงที่เดิมทั้ง 3 รอบของการวนลูป เนื่องจากระบบใช้พื้นที่ใน Data Segment ตำแหน่งเดิมตลอดอายุของโปรแกรม
- **ที่อยู่ของตัวแปรข้ามการรัน:** มีการเปลี่ยนแปลงทุกครั้งที่รันใหม่ เนื่องจาก Ubuntu ตั้งค่าให้ gcc สร้างไบนารีแบบ PIE (Position-Independent Executable) เป็นค่าเริ่มต้น ทำงานร่วมกับกลไก ASLR (Address Space Layout Randomization) ของระบบปฏิบัติการ ซึ่งจะสุ่มฐานตำแหน่งของโปรแกรมใหม่ทุกครั้งที่ถูกโหลดขึ้นหน่วยความจำ

## 1.3 Step 2-3: โค้ดโปรแกรมที่ลบคำว่า static ออก

ทำการแก้ไขโค้ดโดยลบคำว่า static ออก ทำให้ตัวแปร y กลายเป็นตัวแปรอัตโนมัติ (int y = 5;)

```

1     while (x > 0)
2     {
3         int y = 5;          /* deleted 'static' keyword */
4         y++;
5         printf("The value of y is %d\n", y);

```

```

6     printf("The address of y is %p\n\n", &y);
7     x--;
8 }

```

Code 3: src/q1\_auto.c ส่วนที่มีการปรับปรุง

### 1.3.1 ผลการทดลอง

ผลการคอมไพล์และรันโปรแกรม 3 ครั้ง:

```

$ gcc -Wall -Wextra -o q1_auto q1_auto.c
$ ./q1_auto # Run 1
The value of y is 6      The address of y is 0x7fff16657ab0
The value of y is 6      The address of y is 0x7fff16657ab0
The value of y is 6      The address of y is 0x7fff16657ab0

$ ./q1_auto # Run 2
The value of y is 6      The address of y is 0x7ffde6d8b730
The value of y is 6      The address of y is 0x7ffde6d8b730
The value of y is 6      The address of y is 0x7ffde6d8b730

$ ./q1_auto # Run 3
The value of y is 6      The address of y is 0x7ffd35c62ec0
The value of y is 6      The address of y is 0x7ffd35c62ec0
The value of y is 6      The address of y is 0x7ffd35c62ec0

```

Code 4: ผลการทดลอง Step 2–3 (รัน 3 ครั้ง)

### 1.3.2 การวิเคราะห์ผลและเปรียบเทียบกับ Step 1

- **ค่าของตัวแปร ( $y = 6, 6, 6$ ):** ตัวแปร  $y$  เป็น Automatic variable บน Stack ซึ่งมีอายุจำกัดเฉพาะในบล็อกของ loop while ในแต่ละรอบคำสั่ง `int y = 5;` จะทำงานเพื่อกำหนดค่าเริ่มต้นใหม่เสมอ ส่งผลให้ `y++` ได้ค่า 6 เท่ากันทุกครั้ง
- **ที่อยู่ของตัวแปรภายในการรันเดียวกัน:** เท่ากันทั้ง 3 รอบ เนื่องจากการทำงานในลูปยังคงอยู่ใน Stack Frame ของฟังก์ชัน `main()` คอมไพเลอร์จึงนำสล็อตหน่วยความจำเดิมบน Stack มาใช้ซ้ำ (Stack slot reuse)
- **ที่อยู่ของตัวแปรข้ามการรัน:** อยู่ในย่าน Stack (`0x7ff...`) ซึ่งต่างจากตัวแปร static ที่อยู่ในย่าน Data Segment (`0x5...`) โดยตำแหน่งจะถูกสุ่มใหม่ทุกครั้งตามกลไก ASLR ของ Stack

Table 3: เปรียบเทียบผลลัพธ์ระหว่างการใช้และไม่ใช้ static

| ประเด็น             | มี static (Step 1)                  | ไม่มี static (Step 2)           |
|---------------------|-------------------------------------|---------------------------------|
| ค่าที่พิมพ์ออก      | 6, 7, 8 (สะสมค่าต่อเนื่อง)          | 6, 6, 6 (เริ่มต้นใหม่ทุกครั้ง)  |
| การกำหนดค่าเริ่มต้น | ทำครั้งเดียวตอนโหลดโปรแกรม          | ทำซ้ำทุกครั้งเมื่อเข้าบล็อก     |
| ตำแหน่งจัดเก็บ      | Data Segment ( <code>.data</code> ) | Stack Segment                   |
| ขอบเขตการมองเห็น    | เฉพาะในบล็อก while                  | เฉพาะในบล็อก while              |
| อายุการใช้งาน       | ตลอด ระยะเวลา การ ทำงาน ของ โปรแกรม | ชั่วคราวเฉพาะช่วงที่อยู่ในบล็อก |

## 1.4 Step 4: การคอมไพล์ด้วยแฟล็ก `-no-pie` และการเปรียบเทียบ

คอมไพล์โปรแกรมทั้งสองเวอร์ชันใหม่โดยระบุแฟล็ก `-no-pie`:

```

$ gcc -no-pie -Wall -Wextra -o q1_static_nopie q1_static.c
$ gcc -no-pie -Wall -Wextra -o q1_auto_nopie q1_auto.c

```

### 1.4.1 ผลการทดลอง

```

----- 3.1 With static (-no-pie) -----
$ ./q1_static_nopie # Run 1, 2, 3: identical across all runs
The value of y is 6 The address of y is 0x404018
The value of y is 7 The address of y is 0x404018
The value of y is 8 The address of y is 0x404018

----- 3.2 Without static (-no-pie) -----
$ ./q1_auto_nopie # Run 1
The value of y is 6 The address of y is 0x7ffe931c00e0
$ ./q1_auto_nopie # Run 2
The value of y is 6 The address of y is 0x7fff0bc434d0
$ ./q1_auto_nopie # Run 3
The value of y is 6 The address of y is 0x7ffdbc93c020

```

Code 5: ผลการทดลอง Step 4 เมื่อใช้แฟล็ก -no-pie

## 1.4.2 วิเคราะห์ความแตกต่างจากเดิมและสาเหตุ

Table 4: เปรียบเทียบผลลัพธ์ระหว่างการคอมไพล์แบบปกติ (PIE) กับ -no-pie

| กรณี         | คอมไพล์ปกติ (PIE)                               | คอมไพล์ด้วย -no-pie                                | ข้อสรุป                  |
|--------------|-------------------------------------------------|----------------------------------------------------|--------------------------|
| มี static    | 0x5c54..., 0x60a5..., 0x5faa... เปลี่ยนทุกครั้ง | 0x404018 เท่ากันทุกครั้ง                           | แตกต่าง ที่อยู่คงที่     |
| ไม่มี static | 0x7fff..., 0x7ffd..., 0x7ffd... เปลี่ยนทุกครั้ง | 0x7ffe..., 0x7fff..., 0x7ffd... ยังเปลี่ยนทุกครั้ง | ไม่แตกต่าง ที่อยู่ยังสุม |
| ค่าของตัวแปร | 6, 7, 8 และ 6, 6, 6                             | 6, 7, 8 และ 6, 6, 6                                | ไม่แตกต่าง               |

- **กรณีมี static (แตกต่าง):** ที่อยู่ของตัวแปรคงที่เป็น 0x404018 เท่ากันทุกครั้งที่รันใหม่ เนื่องจากแฟล็ก -no-pie สั่งให้ Linker สร้างไฟล์ Executable แบบตำแหน่งคงที่ (ELF EXEC) ซึ่งระบุ Virtual Address ที่แน่นอนตั้งแต่ขั้นตอนเชื่อมโยง เคอร์เนลจึงโหลด Data Segment ลงที่เดิมเสมอ
- **กรณีไม่มี static (ไม่แตกต่าง):** ที่อยู่ของตัวแปรยังคงสุมเปลี่ยนในย่าน 0x7ff... เพราะตัวแปรอัตโนมัติอยู่บน Stack ซึ่งถูกจัดสรรและสุมตำแหน่งโดยตรงจากกลไก ASLR ของเคอร์เนล โดยแฟล็ก -no-pie มีผลเฉพาะการจัดวางตัวโปรแกรมและข้อมูลใน Data Segment เท่านั้น ไม่ได้ปิดการสุมตำแหน่งของ Stack
- **ค่าของตัวแปร:** คงเดิมทั้งสองกรณี เนื่องจากแฟล็ก -no-pie ควบคุมเฉพาะการจัดวางหน่วยความจำในระดับระบบปฏิบัติการ ไม่ส่งผลกระทบต่อตรรกะการประมวลผลของภาษา C

## 1.5 สรุปผลการทดลองข้อ 1

1. คีย์เวิร์ด static ในบล็อกทำหน้าที่เปลี่ยนอายุการใช้งานของตัวแปรให้คงอยู่ตลอดการทำงานของโปรแกรม โดยย้ายตำแหน่งจัดเก็บจาก Stack ไปยัง Data Segment ทำให้สามารถเก็บค่าสะสมข้ามการทำงานของลูปได้
2. ตัวแปรอัตโนมัติถูกจัดสรรบน Stack และสร้างใหม่ทุกครั้งที่เข้าสู่บล็อกคำสั่ง แม้ที่อยู่ภายในการรันเดียวกันจะดูเหมือนเดิม แต่เกิดจากการใช้พื้นที่สล็อตเดิมบน Stack Frame ซ้ำ
3. แฟล็ก -no-pie ปิดการสุมตำแหน่งเฉพาะส่วนของตัวโปรแกรมและ Data Segment เท่านั้น ไม่สามารถปิดการสุมตำแหน่งของ Stack ได้

## 2 ข้อ 2: Extern Storage Class

### 2.1 หลักการพื้นฐาน: การนิยาม (Definition) และการประกาศ (Declaration)

- **การนิยาม (Definition):** เช่น `int x = 20;` ที่อยู่นอกฟังก์ชัน เป็นการสั่งให้ระบบจองพื้นที่หน่วยความจำขนาด 4 ไบต์จริงใน Data Segment โดยในโปรแกรมหนึ่งจะมีนิยามของตัวแปรโกลบอลชื่อเดียวกันได้เพียงแห่งเดียว

- **การประกาศ (Declaration):** เช่น `extern int x;` เป็นการแจ้งคอมพิวเตอร์ว่ามีตัวแปรชื่อนี้และชนิดนี้อยู่ เพื่อให้สามารถอ้างอิงได้ โดยไม่มีการจองพื้นที่หน่วยความจำใหม่

Table 5: เปรียบเทียบการนิยามกับการประกาศตัวแปร

| ประเด็น               | การนิยาม ( <code>int x = 20;</code> นอกฟังก์ชัน) | การประกาศ ( <code>extern int x;</code> ) |
|-----------------------|--------------------------------------------------|------------------------------------------|
| การทำงาน              | จองเนื้อที่จริง 4 ไบต์ใน <code>.data</code>      | ไม่มีการจองเนื้อที่หน่วยความจำ           |
| จำนวนครั้งที่เขียนได้ | 1 ครั้งในทั้งโปรแกรม                             | ซ้ำได้หลายจุดตามที่ต้องการอ้างอิง        |
| ขั้นตอนที่จัดการ      | คอมไพเลอร์จัดสรรพื้นที่                          | Linker ทำการเชื่อมโยงสัญลักษณ์           |

## 2.2 Step 1: โค้ดโปรแกรมที่มี `extern` ทั้งสองฟังก์ชัน

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int x = 20;
5
6 void display()
7 {
8     extern int x;
9     printf(" Display value of x: %d\n", x);
10    printf(" Address of x (in display function): %p\n\n", &x);
11 }
12
13 void main()
14 {
15     extern int x;
16     printf(" Print value of x: %d\n", x);
17     printf(" Address of x (in main function): %p\n\n", &x);
18
19     display();
20 }

```

Code 6: `src/q2_extern.c` โค้ดเริ่มต้นตามใบงาน

### 2.2.1 ผลการทดลอง

```

$ gcc -Wall -Wextra -o q2_extern q2_extern.c
$ ./q2_extern                                # Run 1
Print value of x: 20                          Address of x (in main function): 0x574a1053e010
Display value of x: 20                        Address of x (in display function): 0x574a1053e010

$ ./q2_extern                                # Run 2
Print value of x: 20                          Address of x (in main function): 0x650dd456f010
Display value of x: 20                        Address of x (in display function): 0x650dd456f010

$ ./q2_extern                                # Run 3
Print value of x: 20                          Address of x (in main function): 0x59201f88d010
Display value of x: 20                        Address of x (in display function): 0x59201f88d010

```

Code 7: ผลการทดลอง Step 1 (รัน 3 ครั้ง)

### 2.2.2 การวิเคราะห์ผลการทดลอง

- **ค่าของตัวแปร ( $x = 20$ ):** แสดงค่า 20 ทั้งใน `main()` และ `display()` เนื่องจากทั้งสองฟังก์ชันอ้างอิงถึงตัวแปรโกลบอล `x` ตัวเดียวกันที่นิยามไว้นอกฟังก์ชัน
- **ตำแหน่งที่อยู่ของตัวแปร:** มีค่าตรงกันทั้งใน `main()` และ `display()` ทุกรอบการรัน ยืนยันว่าเป็นการเข้าถึงพื้นที่หน่วยความจำเดียวกันใน Data Segment
- **ตำแหน่งที่อยู่ซ้ำการรัน:** เปลี่ยนแปลงไปทุกครั้งที่รันใหม่ เนื่องจากโปรแกรมคอมไพล์แบบ PIE และทำงานภายใต้ ASLR ของระบบปฏิบัติการ

## 2.3 Step 2-3: โค้ดโปรแกรมที่ลบคำว่า extern ในฟังก์ชัน main()

ทำการแก้ไขตามข้อกำหนดโดยลบคำว่า extern ใน main() ออก (กลายเป็น int x;) ขณะที่ใน display() ยังคงมี extern ตามเดิม

```
1 void main()
2 {
3     int x;                      /* deleted 'extern' keyword: local variable */
4     printf(" Print value of x: %d\n", x);
5     printf(" Address of x (in main function): %p\n\n", &x);
6
7     display();
8 }
```

Code 8: src/q2\_noextern.c ส่วนที่มีการปรับปรุง

### 2.3.1 ผลการทดลอง

การคอมไพล์และการรันโปรแกรม 3 ครั้ง:

```
$ gcc -Wall -Wextra -o q2_noextern q2_noextern.c
q2_noextern.c: In function 'main':
q2_noextern.c:22:5: warning: 'x' is used uninitialized [-Wuninitialized]
22 |     printf(" Print value of x: %d\n", x);

$ ./q2_noextern                                # Run 1
Print value of x: 32767      Address of x (in main function): 0x7ffffbd75e6e4
Display value of x: 20      Address of x (in display function): 0x64d04c4a1010

$ ./q2_noextern                                # Run 2
Print value of x: 32767      Address of x (in main function): 0x7ffff6f1a5714
Display value of x: 20      Address of x (in display function): 0x5595b064f010

$ ./q2_noextern                                # Run 3
Print value of x: 32766      Address of x (in main function): 0x7ffe04a25a14
Display value of x: 20      Address of x (in display function): 0x579c5bb0f010
```

Code 9: ผลการทดลอง Step 2-3 (รัน 3 ครั้ง)

### 2.3.2 การวิเคราะห์ผลและเปรียบเทียบกับ Step 1

- **การเกิด Variable Shadowing:** เมื่อลบคำว่า extern ออก บรรทัด int x; ใน main() จะเปลี่ยนสถานะจากการประกาศเป็นการนิยามตัวแปรอัตโนมัติตัวใหม่บน Stack ทำให้ชื่อตัวแปร x ระดับโลคอลเข้าไปดบังตัวแปรโกลบอล x ภายในขอบเขตของ main()
- **ค่าของ x ใน main():** แสดงค่าขยะ (Garbage value เช่น 32767) เนื่องจากเป็นตัวแปรอัตโนมัติบน Stack ที่ไม่ได้กำหนดค่าเริ่มต้น คอมไพเลอร์จึงแจ้งเตือน -Wuninitialized
- **ค่าของ x ใน display():** ยังคงแสดงค่า 20 ถูกต้องตามเดิม เนื่องจากในฟังก์ชัน display() ไม่ได้ถูกแก้ไข จึงยังคงอ้างอิงและเข้าถึงตัวแปรโกลบอลใน Data Segment ตามปกติ
- **ตำแหน่งที่อยู่:** ที่อยู่ใน main() เป็นย่าน Stack (0x7ff...) ในขณะที่ที่อยู่ใน display() เป็นย่าน Data Segment (0x5... / 0x6...) แสดงให้เห็นอย่างชัดเจนว่าเป็นตัวแปรคนละตัวกัน

Table 6: เปรียบเทียบผลลัพธ์ระหว่างมีและไม่มี extern ใน main()

| ประเด็น                 | มี extern int x; ใน main        | ลบออกเป็น int x; ใน main                |
|-------------------------|---------------------------------|-----------------------------------------|
| ความหมายของบรรทัด       | การประกาศ (อ้างอิงตัวแปรโกลบอล) | การนิยาม (สร้างตัวแปรโลคอลใหม่บน Stack) |
| ค่าที่พิมพ์ใน main()    | 20                              | ค่าขยะ (เช่น 32767)                     |
| ค่าที่พิมพ์ใน display() | 20                              | 20 (ไม่ได้รับผลกระทบ)                   |
| ที่อยู่ของตัวแปร        | ตรงกันทั้งสองฟังก์ชัน (.data)   | คนละตำแหน่ง (main อยู่บน Stack)         |
| การแจ้งเตือนคอมไพเลอร์  | ไม่มี                           | 'x' is used uninitialized               |

## 2.4 Step 4: การคอมไพล์ด้วยแฟล็ก -no-pie และการเปรียบเทียบ

คอมไพล์โปรแกรมทั้งสองเวอร์ชันด้วยแฟล็ก -no-pie:

```

----- 3.1 With extern in both functions (-no-pie) -----
$ ./q2_extern_nopie                # Run 1, 2, 3: identical across all runs
Print value of x: 20                Address of x (in main function): 0x404018
Display value of x: 20              Address of x (in display function): 0x404018

----- 3.2 Without extern in main() (-no-pie) -----
$ ./q2_noextern_nopie              # Run 1
Print value of x: 32767              Address of x (in main function): 0x7fff29c5a044
Display value of x: 20              Address of x (in display function): 0x404020
$ ./q2_noextern_nopie              # Run 2
Print value of x: 32766              Address of x (in main function): 0x7ffead28b514
Display value of x: 20              Address of x (in display function): 0x404020
$ ./q2_noextern_nopie              # Run 3
Print value of x: 32765              Address of x (in main function): 0x7ffd1d346e74
Display value of x: 20              Address of x (in display function): 0x404020

```

Code 10: ผลการทดลอง Step 4 เมื่อใช้แฟล็ก -no-pie

### 2.4.1 วิเคราะห์ความแตกต่างจากเดิมและสาเหตุ

Table 7: เปรียบเทียบผลลัพธ์ของข้อ 2 ระหว่างการคอมไพล์ปกติกับ -no-pie

| ตัวแปรที่พิจารณา     | คอมไพล์ปกติ (PIE)            | คอมไพล์ด้วย -no-pie       | ข้อสรุป                  |
|----------------------|------------------------------|---------------------------|--------------------------|
| Global x (มี extern) | เปลี่ยนตำแหน่งทุกครั้งที่รัน | 0x404018 คงที่ทุกรอบ      | แตกต่าง ที่อยู่คงที่     |
| Global x ใน display  | เปลี่ยนตำแหน่งทุกครั้งที่รัน | 0x404020 คงที่ทุกรอบ      | แตกต่าง ที่อยู่คงที่     |
| Local x ใน main      | 0x7fff... เปลี่ยนทุกครั้ง    | 0x7fff... เปลี่ยนทุกครั้ง | ไม่แตกต่าง ที่อยู่ยังสุม |
| ค่าของตัวแปร         | 20 และค่าขยะ                 | 20 และค่าขยะ              | ไม่แตกต่าง               |

- ตัวแปรโกลบอล (แตกต่าง):** ที่อยู่ของตัวแปรโกลบอลทั้งใน q2\_extern\_nopie และในฟังก์ชัน display() ของ q2\_noextern\_nopie มีตำแหน่งคงที่เดิมเสมอ (0x404018 และ 0x404020) เนื่องจากอยู่ใน Data Segment ของไบนารีตำแหน่งคงที่
- ตัวแปรโลคอลใน main() (ไม่แตกต่าง):** ที่อยู่ยังคงสุมเปลี่ยนทุกครั้งที่รันใหม่ เนื่องจากตัวแปรตั้งอยู่บน Stack ซึ่งยังคงถูกสุมด้วยกลไก ASLR ของเคอร์เนล
- ค่าของตัวแปร:** ยังคงได้ค่าเดิม คือ 20 สำหรับตัวแปรโกลบอล และค่าขยะสำหรับตัวแปรโลคอลที่ไม่ได้กำหนดค่าเริ่มต้น

## 2.5 สรุปผลการทดลองข้อ 2

- คำว่า extern เป็นการประกาศความจำนองเพื่ออ้างอิงตัวแปรโกลบอลที่มีการนิยามไว้แล้ว ไม่มีการจองหน่วยความจำซ้ำซ้อน
- การลบ extern ออกจากตัวแปรในฟังก์ชันจะทำให้เกิดการนิยามตัวแปรโลคอลตัวใหม่บน Stack และเกิดการบดบัง (Shadowing) ตัวแปรโกลบอลเดิม
- แฟล็ก -no-pie ทำให้ตำแหน่งของตัวแปรโกลบอลใน Data Segment มีค่าคงที่ แต่ไม่มีผลต่อตัวแปรอัตโนมัติบน Stack ที่ยังคงถูกสุมโดย ASLR

## 3 ข้อ 3: การจัดสรรหน่วยความจำด้วย malloc, realloc และ free

ตามข้อกำหนดของโจทย์ การทดลองในข้อนี้ทั้งหมดจะคอมไพล์แบบปกติ (without -no-pie) ทำให้ไบนารีทั้งหมดมีคุณสมบัติเป็น PIE ภายใต้อาซาร์

### 3.1 หลักการพื้นฐานของการจัดสรรหน่วยความจำ (Stack vs Heap)

- **Stack Segment:** หน่วยความจำที่คอมพิวเตอร์และระบบปฏิบัติการจัดการให้อัตโนมัติ ใช้เก็บ Local variables และ Stack frame ของฟังก์ชัน ขนาดต้องทราบแน่นอนในขณะคอมไพล์
- **Heap Segment:** หน่วยความจำที่โปรแกรมเมอร์สามารถขอจัดสรร (Allocate) และคืนพื้นที่ (Free) ได้เองแบบพลวัต (Dynamic) ขณะโปรแกรมกำลังทำงานผ่านฟังก์ชันระบบ
- **พอยน์เตอร์กับเป้าหมายที่ชี้:** ตัวแปรพอยน์เตอร์เอง (&a) เป็นตัวแปรโลคอลที่อยู่บน Stack ขนาด 8 ไบต์ (บนระบบ 64-bit) ขณะที่ข้อมูลที่พอยน์เตอร์ชี้ไป (a) จะตั้งอยู่บน Heap

Table 8: เปรียบเทียบย่านหน่วยความจำ Stack, Heap และ Data

| ย่านหน่วยความจำ | ช่วงที่อยู่ (ตัวอย่าง) | กลไกการจอง                 | การคืนพื้นที่                   |
|-----------------|------------------------|----------------------------|---------------------------------|
| Data Segment    | 0x5555...              | กำหนดขนาดคงที่ตอน Link     | คืนอัตโนมัติเมื่อสิ้นสุดโปรแกรม |
| Heap            | 0x5555... / 0x6...     | จัดสรรแบบพลวัตด้วย malloc  | คืนด้วย free โดยโปรแกรมเมอร์    |
| Stack           | 0x7fff...              | จองอัตโนมัติในเฟรมฟังก์ชัน | คืนอัตโนมัติเมื่อออกจากฟังก์ชัน |

### 3.2 Step 1–3: ผลการทดลองและการวิเคราะห์โค้ดเริ่มต้น

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void main()
5  {
6      int *a;
7      float *b;
8      int c[10];
9
10     printf("\nAddress of Pointer\n");
11     printf(">>>p\n", &a);
12     printf(">>>p\n", &b);
13
14     printf("\nEffective Address\n");
15     printf(">>>p\n", a);
16     printf(">>>p\n", b);
17
18     a = (int*) malloc(10 * sizeof(int));
19     printf("\nAfter malloc Pointer a\n");
20     printf(">>>p\n", a);
21     printf(">>>p\n", &a[0]);
22     printf(">>>p\n", &a[9]);
23
24     printf("\nArray c\n");
25     printf(">>>p\n", c);
26     printf(">>>p\n", &c[0]);
27     printf(">>>p\n", &c[9]);
28
29     /* b = (float*) malloc(10 * sizeof(float));
30     printf("\nAfter malloc Pointer b\n");
31     printf(">>>p\n", b);
32     printf(">>>p\n", &b[0]);
33     printf(">>>p\n", &b[9]);
34     */
35
36     a = (int*) realloc(a, 1000 * sizeof(int));
37     printf("\nAfter realloc Pointer a\n");
38     printf(">>>p\n", a);
39     printf(">>>p\n", &a[0]);
40     printf(">>>p\n", &a[9]);
41     printf(">>>p\n", &a[999]);
42
43     free(a);
44     // free(b);
45 }

```

Code 11: src/q3\_a1loc.c โค้ดเริ่มต้นตามใบงาน (ยังคอมเมนต์ส่วนของ b ไว้)

### 3.2.1 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q3_alloc q3_alloc.c
$ ./q3_alloc                                # Run 1                                # Run 2

Address of Pointer      >>0x7ffd2a83f270      >>0x7ffdf5719350
                        >>0x7ffd2a83f278      >>0x7ffdf5719358

Effective Address       >>(nil)              >>(nil)
                        >>(nil)              >>(nil)

After malloc Pointer a   >>0x610cfc37b4b0      >>0x5acd8316e4b0
                        >>0x610cfc37b4b0      >>0x5acd8316e4b0
                        >>0x610cfc37b4d4      >>0x5acd8316e4d4

Array c                 >>0x7ffd2a83f280      >>0x7ffdf5719360
                        >>0x7ffd2a83f280      >>0x7ffdf5719360
                        >>0x7ffd2a83f2a4      >>0x7ffdf5719384

After realloc Pointer a  >>0x610cfc37b4b0      >>0x5acd8316e4b0
                        >>0x610cfc37b4b0      >>0x5acd8316e4b0
                        >>0x610cfc37b4d4      >>0x5acd8316e4d4
                        >>0x610cfc37c44c      >>0x5acd8316f44c
```

Code 12: ผลการทดลอง Step 1-2 (รัน 2 ครั้งเพื่อเปรียบเทียบระยะห่าง)

### 3.2.2 การวิเคราะห์ตามแต่ละส่วนของผลลัพธ์ (Step 3)

#### 1. Address of Pointer (&a, &b):

- **ความหมาย:** คือตำแหน่งที่อยู่ของ ตัวแปรพอยน์เตอร์เอง
- **เหตุผล:** เนื่องจาก a และ b เป็นตัวแปรโลคอล จึงได้รับการจัดสรรไว้บน Stack (0x7ff...) โดยมีระยะห่างระหว่างกันเท่ากับ  $0x278 - 0x270 = 8$  ไบต์ ซึ่งตรงกับขนาดของพอยน์เตอร์บนสถาปัตยกรรม 64-bit พอดี (แม้ b จะเป็น float\* แต่พอยน์เตอร์ทุกชนิดบน 64-bit มีขนาด 8 ไบต์เท่ากัน)

#### 2. Effective Address (a, b ก่อนการจัดสรร):

- **ความหมาย:** คือ ค่าที่ถูกเก็บอยู่ภายในตัวแปรพอยน์เตอร์ (ตำแหน่งปลายทางที่ชี้ไป)
- **เหตุผล:** เนื่องจากก่อนเรียก malloc ตัวแปรทั้งสองยังไม่ได้รับการกำหนดค่าเริ่มต้น จึงมีสถานะเป็น Uninitialized Pointer ซึ่งในสภาพแวดล้อมนี้ระบบแสดงผลเป็น (nil) (ค่า 0) จากการที่หน้าหน่วยความจำใหม่ของ Stack ถูกเคอร์เนลล้างเป็นศูนย์ไว้

#### 3. After malloc Pointer a:

- **ความหมาย:** ตำแหน่งเริ่มต้นของบล็อกหน่วยความจำบน Heap ที่ได้รับการจัดสรรขนาด 40 ไบต์ ( $10 \times \text{sizeof(int)}$ )
- **เหตุผล:** a และ &a[0] มีค่าตรงกัน (0x610cfc37b4b0) เนื่องจากชื่ออาร์เรย์/พอยน์เตอร์ชี้ไปที่จุดเริ่มต้นของสมาชิกตัวแรก และ &a[9] มีค่าห่างออกไป  $0x4d4 - 0x4b0 = 0x24 = 36$  ไบต์ ซึ่งตรงกับ  $9 \times 4$  ไบต์ แสดงว่าข้อมูลถูกจัดวางเรียงต่อกันในหน่วยความจำ

#### 4. Array c:

- **ความหมาย:** ตำแหน่งของอาร์เรย์ขนาด 10 สมาชิกที่จองบน Stack
- **เหตุผล:** c และ &c[0] มีตำแหน่งตรงกันที่ 0x7ffd2a83f280 และ &c[9] อยู่ถัดไป 36 ไบต์ที่ 0x7ffdf5719384 แต่ตำแหน่งจัดเก็บอยู่ในย่าน Stack (0x7ff...) ซึ่งอยู่คนละส่วนกับ Heap ของ a อย่างสิ้นเชิง

#### 5. After realloc Pointer a:

- **ความหมาย:** ตำแหน่งของหน่วยความจำหลังขยายขนาดเป็น 4000 ไบต์ ( $1000 \times \text{sizeof(int)}$ )
- **เหตุผล:** ในรอบนี้ ที่อยู่ของ a ยังคงเป็น ที่อยู่เดิมทุกประการ (0x610cfc37b4b0) เนื่องจากก่อนของ a อยู่ท้ายสุดของ Heap และพื้นที่ด้านหลังยังว่างอยู่ ตัวจัดสรรจึงสามารถขยายขนาดบล็อกเดิมต่อท้ายได้ทันที (In-place growth) โดยไม่ต้องย้ายที่อยู่ และ &a[999] อยู่ห่างจาก a เท่ากับ  $999 \times 4 = 3996$  ไบต์ ( $0x610cfc37c44c - 0x610cfc37b4b0 = 0xf9c$ )

### 3.3 Step 4–6: ผลการทดลองเมื่อเปิดคอมเมนต์ของตัวแปร b

เปิดคอมเมนต์ใน 2 ตำแหน่งตามที่โจทย์ระบุ ได้แก่ บล็อกคำสั่ง malloc ของ b และคำสั่ง free(b) ที่ท้ายโปรแกรม

```

1  /* uncommented malloc(b) and free(b) */
2  b = (float*) malloc(10 * sizeof(float));
3  printf("\nAfter malloc Pointer b\n");
4  printf(">>p\n", b);
5  printf(">>p\n", &b[0]);
6  printf(">>p\n", &b[9]);
7
8  a = (int*) realloc(a, 1000 * sizeof(int));
9  ...
10 free(a);
11 free(b);

```

Code 13: src/q3\_alloc\_b.c ส่วนที่มีการเปิดคอมเมนต์

#### 3.3.1 ผลการทดลอง

```

$ gcc -Wall -Wextra -o q3_alloc_b q3_alloc_b.c
$ ./q3_alloc_b
# Run 1
# Run 2

Address of Pointer    >>0x7ffd3f507340    >>0x7ffe68018760
                    >>0x7ffd3f507348    >>0x7ffe68018768

Effective Address    >>(nil)           >>(nil)
                    >>(nil)           >>(nil)

After malloc Pointer a  >>0x6124eb5724b0    >>0x5bf8495814b0
                    >>0x6124eb5724b0    >>0x5bf8495814b0
                    >>0x6124eb5724d4    >>0x5bf8495814d4

Array c              >>0x7ffd3f507350    >>0x7ffe68018770
                    >>0x7ffd3f507350    >>0x7ffe68018770
                    >>0x7ffd3f507374    >>0x7ffe68018794

After malloc Pointer b  >>0x6124eb5724e0    >>0x5bf8495814e0
                    >>0x6124eb5724e0    >>0x5bf8495814e0
                    >>0x6124eb572504    >>0x5bf849581504

After realloc Pointer a >>0x6124eb572510    >>0x5bf849581510
                    >>0x6124eb572510    >>0x5bf849581510
                    >>0x6124eb572534    >>0x5bf849581534
                    >>0x6124eb5734ac    >>0x5bf8495824ac

```

Code 14: ผลการทดลอง Step 4–5 (รัน 2 ครั้งหลังเปิดคอมเมนต์)

#### 3.3.2 การวิเคราะห์และเปรียบเทียบกับรอบก่อนหน้า (Step 6)

คำตอบสำหรับโจทย์ Step 6: ผลลัพธ์ที่ได้ แตกต่างจากรอบก่อนหน้าอย่างมีนัยสำคัญ โดยเฉพาะ ที่อยู่ของอาร์เรย์ a หลังการเรียก realloc()

Table 9: เปรียบเทียบพฤติกรรมของ realloc(a) ระหว่างสองกรณี

| การดำเนินการ      | ไม่มีการจอง b (Step 1)            | มีการจอง b คั่นกลาง (Step 4)            |
|-------------------|-----------------------------------|-----------------------------------------|
| a หลัง malloc     | 0x610cfc37b4b0                    | 0x6124eb5724b0                          |
| b หลัง malloc     | (ไม่มีการจอง)                     | 0x6124eb5724e0 (อยู่ติดท้ายก้อน a)      |
| a หลัง realloc    | 0x610cfc37b4b0 (ที่อยู่เดิม)      | 0x6124eb572510 (เปลี่ยนที่อยู่ใหม่)     |
| พฤติกรรมการจัดสรร | ขยายพื้นที่เดิมต่อท้าย (In-place) | ย้ายบล็อกไปยังที่ใหม่บน Heap (Relocate) |

สาเหตุที่อยู่ของ a เปลี่ยนไป:

- ใน Step 1 มีการจองเฉพาะ a ทำให้พื้นที่ถัดจากก้อนของ a ยังคงเป็นพื้นที่ว่าง ตัวจัดสรรจึงสามารถขยายขนาดในตำแหน่งเดิมได้ทันที
- ใน Step 4 มีการเรียก malloc ให้กับ b ก่อนการเรียก realloc(a) ทำให้ก้อนหน่วยความจำของ b ถูกจัดสรรลงไปขวางพื้นที่ต่อท้ายก้อนของ a พอดี (0x...24e0 ต่อจาก 0x...24b0)
- เมื่อสั่งขยายขนาด a เป็น 4000 ไบต์ ตัวจัดสรรไม่สามารถขยายขนาดต่อท้ายที่เดิมได้เพราะติดพื้นที่ของ b จึงจำเป็นต้องค้นหาพื้นที่ว่างก้อนใหม่ที่มีขนาดใหญ่เพียงพอ ย้ายข้อมูลเดิมจากก้อนเก่าไปยังที่อยู่ใหม่ (0x...2510) แล้วจึงปลดปล่อยพื้นที่ก้อนเดิม

### 3.4 สรุปผลการทดลองข้อ 3

1. ตัวแปรพอยน์เตอร์จะถูกจัดสรรบน Stack เสมอ (ขนาด 8 ไบต์บน 64-bit) ขณะที่พื้นที่หน่วยความจำที่ได้รับจาก malloc จะอยู่บน Heap Segment
2. ฟังก์ชัน realloc จะพยายามขยายขนาดบล็อกหน่วยความจำในตำแหน่งเดิม (In-place) หากพื้นที่ด้านหลังยังคงว่างอยู่
3. หากพื้นที่ด้านหลังถูกจัดสรรให้ตัวแปรอื่นไปแล้ว realloc จะจัดสรรบล็อกใหม่ ย้ายข้อมูลเดิม และคืนที่อยู่ใหม่ ส่งผลให้พอยน์เตอร์เดิมถูกเปลี่ยนตำแหน่ง ซึ่งโปรแกรมเมอร์ต้องอัปเดตพอยน์เตอร์และไม่ควรใช้อ้างอิงที่อยู่เดิม
4. การใช้งาน free ต้องกระทำอย่างถูกต้องตามคู่ของ malloc เพื่อป้องกันปัญหาหน่วยความจำรั่วไหล (Memory Leak) หรือข้อผิดพลาดจากการคืนพื้นที่ที่ไม่ได้จัดสรร